CSCI160 Computer Science I

Lab: Functional decomposition, docstrings, doctests and complete date validation

Objectives

- Function definition and calling
- Function documentation using type hints
- Writing documentation strings and doctests
- Writing functions that call other functions
- Clear function naming for contextual use
- Documentation via comments and citations

NOTE: Lack of documentation and thorough AI citations will be egregiously punished

Leap Years

A year in the modern Gregorian Calendar consists of 365 days. In reality, the earth takes longer to rotate around the sun. To account for the difference in time, every 4 years, a leap year takes place. A leap year is when a year has 366 days: An extra day, February 29th. The requirements for a given year to be a leap year are:

1. The year must be divisible by 4
2. If the year is a century year (1700, 1800, etc.), the year must be evenly divisible by 400
3. Some example leap years are 1600, 1712, and 2016.

TASK ONE: In the file **`dates.py`** write the function

`is_leap_year(year: int) -> bool`

that accepts a year and returns **Boolean True or False** (not String 'True' or 'False') depending on the Leap Year tests.

Function requirements

1) is_leap_year should NOT print anything from inside the function. It simply returns the boolean values: True or False

2) is_leap_year should NOT ask for input. All input needs to happen from the application file and be passed into the function. The function should accept the year in question as an argument.

Docstrings, doctests and type hints are REQUIRED

Include a docstring of the format

```
Tests year for leap year status

Parameters:
    year (int): The year

Returns:
    True (Boolean)   if year is a leap year
    False (Boolean)  if year is not a leap year

Examples/Doctests:
    >>> is_leap_year(1900)
    False

    >>> is_leap_year(2000)
    True

    >>> is_leap_year(2020)
    True

    >>> is_leap_year(2021)
    False
```

Ensure your code works by performing the following

1. Run 5 test cases at the bottom of the `dates.py` script file. The test cases should be different from the doctests inside the function docstring. Print the results
    1. Running **`python dates.py`** from the terminal should clearly show the test case output
    2. Importing the **`dates`** module should not cause the test cases to run
2. Execute the function's doctests with verbose output (-v)
    1. Running **`python -m doctest -v dates.py`** from the terminal should clearly show the doctest output
3. When you submit your work, include a screen shot of the doctests passing **with verbose output.**

TASK TWO: Still in `dates.py` write the function

`is_valid_format(date: str) -> bool`

This function will accept a Date as a String argument and return Boolean True or False based on the following rules

The format of a Date is correct if the following are true

1. Must be MM/DD/YYYY. Single digit months or days should be padded with a zero
2. There are two "/" characters
3. Month field is 2 characters
4. Day field is 2 characters
5. Year field is 4 characters
6. All three fields are integers

Function requirements

1) is_valid_format should NOT print anything from inside the function. It simply returns a Boolean signifying the passing of the 6 tests listed above.

2) is_valid_format should NOT ask for input. All input needs to happen in the application file and be passed into the function. The function should accept the string in question as an argument.

Include a docstring in a format similar to what has been shown in the lab.

Include 4 doctest examples. 2 for False cases, 2 for True cases

MILESTONE 2: Prove your code works by performing the following

1. Run 5 test cases at the bottom of the script file. The test cases should be different from the doctests inside the function docstring. Print the results
2. Execute the function's doctests with verbose output. This run will execute all tests in the file
3. **When you submit your work, include a screen shot of the doctests passing with verbose output.**

TASK THREE: Still in `dates.py` write the function

`split_date(date: str) -> list[int]`

that accepts a date in `"MM/DD/YYYY"` format and returns a 3 element list of integers where

```
first  element is MM   converted to an integer
second element is DD   converted to an integer
third  element is YYYY converted to an integer
```

```
d = split_date("01/02/2020")

d == [1, 2, 2020]
```

Include a docstring in a format similar to what has been shown in the lab.

Include 3 doctest examples.

MILESTONE 3: Prove your code works by performing the following

4. Run 5 test cases at the bottom of the script file. The test cases should be different from the doctests inside the function docstring. Print the results
5. Execute the function's doctests with verbose output. This run will execute all tests in the file
6. **When you submit your work, include a screen shot of the doctests passing with verbose output.**

TASK FOUR: Still in `dates.py` write the following collection of functions. Include docstrings and doctests for all!

`is_valid_month(month: int) -> bool:`
that accepts a month as an integer and returns True if it's in the range {1 – 12}. Return False otherwise

`is_valid_day(month: int, day: int, year: int) -> bool:`
that accepts 3 integers representing the month, day and year and returns True if the day is valid for that month. Return False otherwise.

This function must incorporate the `is_leap_year` function from an earlier task. If it is a leap year then February allows 29 days.

```
is_valid_day(2, 29, 2020) should return True
is_valid_day(2, 29, 2021) should return False
```

Restrictions: You are not allowed to write a giant wall of if statements for this. Define a dictionary that equates the month to the number of days in the month and use that. Define this dictionary at `module level` (near the top of the file, outside of any function definition).

`full_date_string(date: str) -> str:`

That accepts a date in “MM/DD/YYYY” format and returns a string in the format

```
month_name day, year
```

Use the months dictionary from earlier to associate days with months without a wall of if statements. If the month names aren’t in there . . . add them. Perhaps the keys are month numbers and the value is a list that contains

```
[“Full month name”, number_of_days_in_month]

full_date_string("02/29/2020") should return 'February 29, 2020'
full_date_string("01/31/2020") should return 'January 31, 2020'
```

`is_valid_date(date: str) -> bool:`

That accepts a date in "MM/DD/YYYY format and returns True if the date string is an **actual valid date** and False otherwise. This function should call all of the functions you wrote above to accomplish its task. A sample sequence could be

1. `is_valid_format` – exit early if the format is invalid
2. `split_date`
3. `is_valid_month` – exit early if the month is invalid
4. `is_valid_day which calls is_leap_year`

The date_string will need to pass through all of those tests in order to prove a date is valid.

Restrictions: You must solve this function by calling all the other functions. You are not allowed to repeat any logic here.

TASK FIVE: Putting it all together.

You have been provided with a text file containing a collection of sample dates. Some of these dates are correct and some are not. Your task is label all the dates as correct or incorrect,

Create a new file called `app.py`

Steps:

1. Import appropriate functions from `dates.py`. You only need
    1. `is_valid_date`
    2. `full_date_string`
    3. Do not import functions that you do not use
2. Open the file `dates.txt` for reading
3. Iterate through the file, stripping each line of the line break character.
4. Pass the stripped date into the `is_valid_date` function

5. If the date is valid display the full date string followed by “is a valid date”

   1. 01/01/2020
   2. January 1, 2020 is a valid date

6. If the date is not valid display the original input followed by “is not a valid date”

   1. 02/29/1900
   2. 02/29/1900 is not a valid date

7. If the first four dates in the file are

   ```
   01/15/2022
   02/29/2020
   02/30/2020
   04/31/2021
   ```

   Then the display when running **`python app.py`** would be:

   ```
   January 15, 2020 is a valid date
   February 29, 2020 is a valid date
   02/30/2020 is not a valid date
   04/31/2021 is not a valid date
   ```

8. For testing purposes I have included an annotated copy of the original text file that shows which dates are problematic. When grading a completely different file will be used.

Submission: Attach `dates.py` and `app.py` to the submission area along with the requested screenshots of doctests being executed and passing.

Check the rubric to ensure you have everything covered. **Documentation is required**